

MERN Full Stack Final Assessment (FA)

Assessment Pattern & Guidelines

1. Assessment Objective

This assessment evaluates your ability to build a complete MERN Full Stack application using:

- MongoDB Atlas
- Express.js
- React.js
- Node.js

The assessment focuses on:

- API integration
- Data validation and sanitization
- MongoDB persistence
- Backend API development
- Frontend integration
- State management
- Authentication handling
- Full-stack application deployment

2. Assessment Overview

You will be provided with:

- A Public API
- A Private API

Your task is to:

1. Obtain a token using your student credentials
2. Use the token to fetch dataset from the private API
3. Validate and sanitize the dataset
4. Store valid data into MongoDB Atlas
5. Build REST APIs using the stored MongoDB data
6. Build a React frontend consuming your backend APIs
7. Deploy both backend and frontend applications
8. Submit GitHub and deployment URLs

3. Assessment Duration

Total Duration: 3 Hours

4. Technology Requirements

Backend

Mandatory:

- Node.js
- Express.js
- MongoDB Atlas
- Mongoose
- Axios or Fetch
- dotenv

Frontend

Mandatory:

- React.js
- React Router
- Context API + Reducer
- Axios or Fetch

5. Step 1 — Obtain Token

Public API Endpoint

POST /public/token

Request Body

```
{
  "studentId": "YOUR_REGISTER_NUMBER",
  "password": "YOUR_PASSWORD"
}
```

Response

```
{
  "token": "JWT_TOKEN"
}
```

6. Step 2 — Fetch Dataset

Private API Endpoint

GET /private/data

Required Header

```
Authorization: Bearer <JWT_TOKEN>
```

7. Dataset Rules

The dataset may contain:

- Missing fields
- Duplicate records
- Invalid values
- Inconsistent formatting
- Null or empty values

Students are expected to:

- Validate records
- Sanitize data
- Normalize values
- Reject invalid records
- Prevent duplicate insertion

8. MongoDB Requirements

Students must:

- Connect MongoDB Atlas
- Create Mongoose models
- Store cleaned records into MongoDB
- Use MongoDB as persistent storage

In-memory caching is NOT allowed.

Dataset Availability Policy

The private dataset server is intended only for initial synchronization.

Students are expected to:

- Fetch the dataset
- Validate and sanitize records
- Persist cleaned records into MongoDB Atlas using the /sync process

Applications must continue functioning correctly using MongoDB-stored data even when the external dataset service is unavailable.

Repeated dependency on the external dataset API for runtime operations is not allowed.

9. Mandatory Backend APIs

You must implement the following APIs.

9.1 Sync Endpoint

POST /sync

Responsibilities:

- Fetch dataset from private API
- Validate and sanitize data
- Insert valid records into MongoDB
- Prevent duplicate insertion

Example Response

```
{
  "success": true,
  "totalFetched": 100,
  "inserted": 82,
  "duplicates": 5,
  "rejected": 13
}
```

9.2 Get All Records

GET /tasks

9.3 Get Record by ID

GET /tasks/:id

9.4 Create Record

POST /tasks

9.5 Update Record

PUT /tasks/:id

9.6 Delete Record

DELETE /tasks/:id

9.7 Filter/Search API

Examples:

GET /tasks?status=pending

GET /tasks?priority=high

GET /tasks/search?q=meeting

9.8 Stats API

GET /stats

The API must dynamically compute:

- Total records
- Completed records
- Pending records
- Priority-wise grouping
- Other required analytics

9.9 Health API

GET /health

Example Response

```
{
  "success": true,
  "database": "connected",
  "documentCount": 82
}
```

10. Standard Response Structure

All APIs must return responses in the following structure.

Success Response

```
{
  "success": true,
  "message": "Operation successful",
  "data": []
}
```

Error Response

```
{
  "success": false,
  "message": "Error message"
}
```

11. Backend Technical Requirements

Your backend must include:

- Express Router
- Async/Await
- Proper error handling
- Middleware usage
- Environment variables
- Mongoose models
- Clean folder structure

12. Recommended Backend Folder Structure

12.1 Backend

```
src/  
├─ controllers/  
├─ middleware/  
├─ models/  
├─ routes/  
├─ services/  
├─ utils/  
├─ app.js  
└─ server.js
```

12.1 Frontend

```
src/  
├─ components/  
├─ context/  
├─ hooks/  
├─ pages/  
├─ reducer/  
├─ router/  
├─ services/  
├─ utils/  
└─ App.jsx
```

13. Frontend Requirements

Your frontend application must:

- Consume your own backend APIs
- Implement protected routes
- Display synchronized MongoDB data



- Implement CRUD operations
- Support filtering and search
- Render dashboard analytics
- Handle loading and error states

14. Mandatory Frontend Features

Authentication

- Login flow
- Token handling
- Protected routing

Dashboard

- Display records dynamically
- Display analytics
- Render summary cards

CRUD Operations

- Add record
- Edit record
- Delete record
- Update status

Filtering & Search

Examples:

- Search by keyword
- Filter by status
- Filter by priority

State Management

Mandatory:

- Context API
- Reducer

15. Frontend Technical Requirements

Your frontend must include:

- Functional Components
- useState

- useEffect
- useContext
- useReducer
- React Router
- Axios or Fetch
- Form validation

16. Frontend Testing Requirements

Students must include:

- Reducer tests
- Utility function tests

Recommended:

- Vitest
OR
- Jest

17. data-testid Requirement

Students must include:

```
<button data-testid="add-task-btn">
```

for interactive frontend elements.

This is mandatory for automated evaluation.

18. Deployment Requirements

Backend Deployment

Recommended:

- Render

Requirements:

- Use process.env.PORT
- Proper environment variables
- MongoDB Atlas connection

Frontend Deployment

Recommended:

- Vercel
- Netlify

Requirements:

- Working routing
- No console errors
- Successful backend integration

19. Submission Requirements

Students must submit:

Backend

- GitHub Repository URL
- Live Backend URL

Frontend

- GitHub Repository URL
- Live Frontend URL

20. Evaluation Criteria

Criteria	Marks
Authentication & Token Flow	10
Dataset Validation & Cleaning	15
MongoDB Persistence	15
CRUD APIs	15
Aggregation APIs	10
Frontend Integration	15
UI/UX + State Management	10
Unit Testing	5
Deployment	5

21. Academic Integrity

The following are strictly prohibited:

- Hardcoding API responses
- Sharing tokens
- Sharing datasets
- Copying code from other students
- Bypassing API integration

Any malpractice may result in disqualification.



22. Important Instructions

- Build both backend and frontend applications.
- Frontend must consume your own backend APIs.
- MongoDB Atlas connection is mandatory.
- Maintain proper folder structure.
- Use environment variables securely.
- Ensure deployment URLs are working before submission.

23. Final Note

This assessment evaluates your ability to engineer a complete full-stack application under realistic software development conditions.

Focus on:

- Clean architecture
- Correct API behavior
- Data validation
- Proper MongoDB usage
- Frontend-backend integration
- Reliable deployment

— All the best —